

Plan Then Retrieve: Reinforcement Learning-Guided Complex Reasoning over Knowledge Graphs

Yanlin Song*
School of Computer Science
Wuhan University
Wuhan, China
songyanlin@whu.edu.cn

Ben Liu*
Ant Group
Hangzhou, China
keli.lb@antgroup.com

Víctor Gutiérrez-Basulto
School of Computer Science and
Informatics
Cardiff University
Cardiff, UK
GutierrezBasultoV@cardiff.ac.uk

Zhiwei Hu
College of Information Science and
Engineering
Shanxi Agricultural University
Taiyuan, China
zhiwei.hu@whu.edu.cn

Qianqian Xie[†]
School of Artificial Intelligence
Wuhan University
Wuhan, China
Center for Language and Information
Research
Wuhan University
Wuhan, China
xieq@whu.edu.cn

Min Peng[†]
School of Artificial Intelligence
Wuhan University
Wuhan, China
Center for Language and Information
Research
Wuhan University
Wuhan, China
pengm@whu.edu.cn

Sophia Ananiadou
School of Computer Science
University of Manchester
Manchester, UK
sophia.ananiadou@manchester.ac.uk

Jeff Z. Pan
ILCC, School of Informatics
University of Edinburgh
Edinburgh, UK
j.z.pan@ed.ac.uk

Abstract

Knowledge Graph Question Answering (KGQA) aims to answer natural language questions by reasoning over structured knowledge graphs (KGs). While large language models (LLMs) have advanced KGQA through their strong reasoning capabilities, existing methods continue to struggle to fully exploit both the rich knowledge encoded in KGs and the reasoning capabilities of LLMs, particularly in complex scenarios. They often assume complete KG coverage and lack mechanisms to judge when external information is needed, and their reasoning remains locally myopic, failing to maintain coherent multi-step planning, leading to reasoning failures even when relevant knowledge exists. We propose **Graph-RFT**, a novel two-stage reinforcement fine-tuning KGQA framework with a “plan-KGsearch-and-Websearch-during-think” paradigm, that enables LLMs to perform autonomous planning and adaptive retrieval scheduling across KG and web sources under incomplete

knowledge conditions. Graph-RFT introduces a chain-of-thought (CoT) fine-tuning method with a customized plan-retrieval dataset activates structured reasoning and resolves the GRPO cold-start problem. It then introduces a novel plan-retrieval guided reinforcement learning process integrates explicit planning and retrieval actions with a multi-reward design, enabling coverage-aware retrieval scheduling. It employs a Cartesian-inspired planning module to decompose complex questions into ordered sub-questions, and logical expression to guide tool invocation for globally consistent multi-step reasoning. This reasoning-retrieval process is optimized with a multi-reward combining outcome and retrieval-specific signals, enabling the model to learn when and how to combine KG and web retrieval effectively. Experiments on multiple KGQA benchmarks demonstrate that Graph-RFT achieves superior performance over strong baselines, even with smaller LLM backbones, and substantially improves complex question decomposition, factual coverage, and tool coordination.

*Equal contribution.

[†]Corresponding Author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnnn.nnnnnnn>

CCS Concepts

• **Computing methodologies** → *Semantic networks*.

Keywords

Knowledge Graph Question Answering, Large Language Model, Reinforcement Learning

ACM Reference Format:

Yanlin Song, Ben Liu, Victor Gutiérrez-Basulto, Zhiwei Hu, Qianqian Xie, Min Peng, Sophia Ananiadou, and Jeff Z. Pan. 2026. Plan Then Retrieve: Reinforcement Learning-Guided Complex Reasoning over Knowledge Graphs. In . ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Knowledge Graph Question Answering (KGQA) aims to answer natural language questions by reasoning over structured knowledge graphs (KGs) that store explicit, factual knowledge in the form of triples. KGQA plays a central role in web-based intelligent systems such as search, recommendation, and social platforms [17, 38, 40–42, 47, 49], where accurate and explainable reasoning over structured data is crucial. Recent advances in large language models (LLMs) have transformed natural language understanding, reasoning, and generation [5, 9, 11, 22, 32]. Leveraging their strong generalization and reasoning capabilities, recent KGQA methods increasingly rely on LLMs, integrating knowledge graphs as structured, verifiable knowledge sources to enhance factual precision and interpretability [13, 21, 27, 33, 36, 45].

Existing LLM-based KGQA approaches fall into two paradigms: **semantic parsing (SP)**, which translates natural questions into executable logical queries (e.g., SPARQL) and runs them over a KG [19, 24, 44], and **retrieval-augmented generation (RAG)**, which retrieves relevant KG triples to condition an LLM’s generation [36, 44]. Despite substantial progress, both paradigms struggle in complex settings for two main reasons. First, they lack a reliable mechanism to judge whether a KG contains the facts necessary to answer a question. SP and RAG methods commonly assume that all required triples are present in the KG, a strong and unrealistic assumption for manually curated graphs. When coverage is incomplete, SP fails to produce correct executable queries, and RAG offers no built-in way to refine retrieval or to decide when to consult external (unstructured) sources. Second, these methods often lack the ability to perform coherent multi-step planning and adaptive retrieval scheduling. For example in Figure 1(2), an LLM may correctly decompose a question into “Which screenwriters wrote Analyze That?” and retrieve “Harold Ramis,” but then fail to continue the planned decomposition and instead reformulates the follow-up as “When was Harold Ramis released?”, confusing the person with a film title. This failure stems from constrained local decision-making and an absence of coherent multi-step planning, which leads to retrieval and reasoning errors even when the KG contains the necessary facts. The comparison for previous work and Graph-RFT is in Table 1.

To address these challenges, we propose Graph-RFT, a novel two-stage reinforcement fine-tuning framework designed to enhance an LLM’s capabilities in autonomous planning and adaptive retrieval scheduling across multiple tools (specifically, KG search and web search) under conditions of incomplete KGs. In the first stage (Section 3.2), we introduce a chain-of-thought (CoT) [39] fine-tuning method that uses a customized plan–retrieval dataset to explicitly train the model to decompose questions, formulate plans, and select retrieval tools. This stage not only activates the model’s reasoning and planning capabilities but also resolves the GRPO [9] cold-start problem, enabling stable and meaningful reinforcement rollouts.

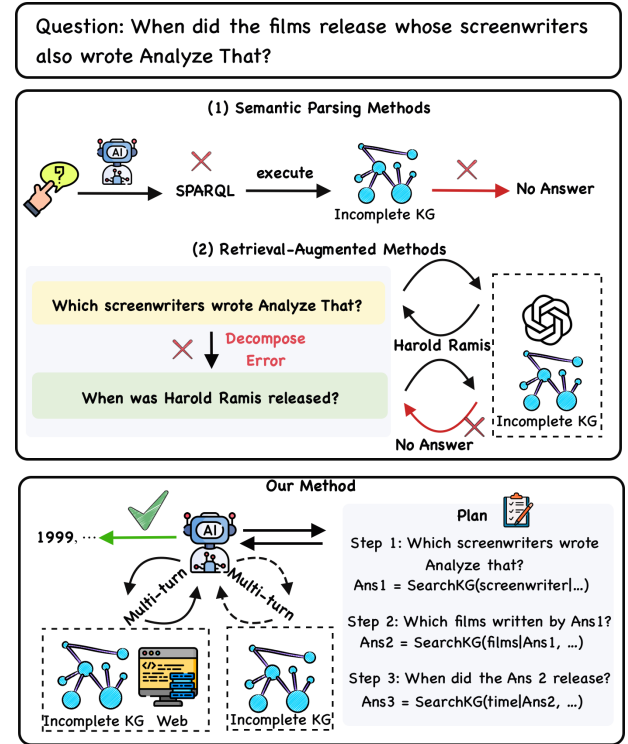


Figure 1: The comparison of Semantic Parsing, Retrieval-Augmented Methods, and Graph-RFT.

In the second stage (Section 3.3), we propose a plan–retrieval guided reinforcement learning method that integrates explicit planning steps and retrieval actions into the model’s reasoning loop, and learns a coverage-aware retrieval policy that dynamically coordinates knowledge graph and web search under incomplete KG conditions. It uses a structured template $\langle \text{plan} \rangle \langle \text{relation_search} \rangle \langle \text{neighbor_search} \rangle \langle \text{web_search} \rangle$, to explicitly let the model to alternate between symbolic planning and retrieval actions. The planning module draws inspiration from Cartesian principles [6], breaking complex questions into logically ordered sub-questions. We further employ logical expressions to represent dependencies among sub-questions and guide the invocation of KG retrieval tools, ensuring globally coherent multi-step reasoning and controllable retrieval flow. For each planned execution step, Graph-RFT invokes both relation-search and neighbor-search tools to perform KG retrieval; when the KG lacks sufficient coverage, the model automatically triggers web search to acquire supplementary unstructured evidence. To optimize the reasoning–retrieval process, we apply a GRPO-based reinforcement learning with a novel multi-reward design: an outcome reward measuring factual correctness and a retrieval-specific reward that evaluates retrieval coverage, precision, and timing. This formulation encourages the model to learn when and how to combine KG and web retrieval, overcoming the limitations of prior SP and RAG methods that assume complete KGs and the absence of coherent multi-step planning.

Our main contributions are as follows:

Table 1: Comparison for main characteristics of previous SP, RAG methods with our Graph-RFT.

Class	Algorithm	Paradigm	Incomplete KG	Global Planning	Decompose Problems	KG Interaction
SP	RGR-KBQA [7]	SFT	×	×	×	×
SP	Rule-KBQA [48]	SFT	×	×	×	×
RAG	PoG [36]	ICL	×	×	✓	✓
RAG	DoG [27]	ICL	×	×	✓	✓
RAG	KG-Agent [14]	SFT	×	×	✓	✓
Our Method	Graph-RFT	SFT & RL	✓	✓	✓	✓

- We introduce Graph-RFT, a novel two-stage reinforcement fine-tuning framework for complex reasoning over incomplete KGs. It introduces the novel CoT-based fine-tuning method to activate planning and reasoning capabilities while resolving the GRPO cold-start issue, and a plan–retrieval guided reinforcement learning method to enable coherent multi-step reasoning and adaptive retrieval scheduling across KG and web sources under incomplete KG conditions.
- We propose a plan–retrieval guided reinforcement learning method with a multi-reward design that integrates graph-retrieval, web-retrieval, and penalty signals. This fine-grained feedback enables the model to learn when and how to combine KG and web retrieval for adaptive, coverage-aware reasoning under incomplete KGs.
- Experimental results across several widely used complex reasoning datasets confirm the superior performance of Graph-RFT against strong baselines, even when utilizing weaker LLM backbones (i.e., the 7B series). Furthermore, comprehensive empirical analyses validate Graph-RFT’s effectiveness across multiple critical aspects, including complex question decomposition planning, the identification of missing factual triples, and the appropriate invocation of various tools.

2 Related Work

Existing methods for improving LLM-based KGQA can be categorized into two main types: Semantic Parsing (SP) methods and Retrieval Augmented (RA) methods.

SP Methods convert natural language questions into structural queries (e.g., SPARQL) that can be executed on KGs to retrieve answers. Early research efforts [12, 25, 43] in semantic parsing often involved generating query graphs, resembling subgraphs of the KG and allow for direct mapping to logical forms. Lan [18] incorporates constraints into the query graph, which effectively narrows the search space and facilitates more flexible query graph generation. Subsequent works [2, 15, 18, 29, 30, 39] utilize sequence-to-sequence models for multi-hop path and SPARQL-expression generation, enhancing the semantic parsing process. Recently, with the emergence of LLMs, various methods unify KGs and LLMs for KGQA. ChatKBQA [24] generates question-aligned logical forms by treating LLMs as semantic parsers and retrieves entities and relations from KGs. RGR-KBQA [7] retrieves factual knowledge from KGs to enhance the semantic understanding capabilities of LLMs and finetune them to generate the logical form. Rule-KBQA [48] guides logical form generation via a two-phase process that induces a rule library and deducts forms via a rule-guided agent. However, these methods depend heavily on the assumption that KGs are

complete, which means no answers can be obtained if information is missing.

RA Methods retrieve relevant KG triple based on input questions, then use these triples to guide LLMs to generate accurate answers, emphasizing the dynamic interaction between KG retrieval and LLM reasoning. ToG [33] employs an explore-and-exploit approach, which can discover the most promising reasoning paths and return the most likely reasoning results. GoG [44] proposes a think-search-generate paradigm, alleviating the incompleteness issue of KGs by using LLMs’ internal knowledge. PoG [36] leverages knowledge reasoning paths as retrieval-augmented inputs for LLMs, thereby alleviating issues related to insufficient domain knowledge and unfaithful reasoning chains. DoG [27] introduces a subgraph-focused mechanism that allows LLMs to attempt intermediate answers after each reasoning step, effectively mitigating the drawbacks of lengthy reasoning trajectories. However, most of these approaches depend on powerful closed-source LLM APIs (e.g., GPT-4 [1]), which results in substantial performance degradation when weaker open-source models are used as backbones. KG-Agent [14] develops an iterative reasoning framework that integrates an LLM, a multifunctional toolbox, a KG-based executor, and a knowledge memory module, enabling smaller LLMs to make decisions autonomously. SymAgent [21] conceptualizes the KG as a dynamic environment and is designed to autonomously and effectively integrate the capabilities of both the LLM and the KG. Graph-R1 [23] proposes lightweight knowledge hypergraph construction and retrieval framed as a multi-turn agent–environment interaction, optimizing the reasoning process via an end-to-end reward mechanism. DynaSearcher [10] utilizes knowledge graphs as external structured knowledge to guide the search process by explicitly modeling entity relationships, thereby ensuring factual consistency in intermediate queries.

3 Methodology

In this section, we introduce Graph-RFT, with autonomous planning and adaptive retrieval capabilities for reasoning over incomplete KGs. The framework is structured around two complementary core stages: (1) CoT Fine-Tuning for Reasoning Activation: we propose a supervised fine-tuning (SFT) method using high-quality CoT plan–retrieval trajectories to activate the model’s structured reasoning and planning abilities while resolving the GRPO cold-start problem. (2) RL-based Reasoning Enhancement: we propose a plan–retrieval guided RL method with a multi-reward design to optimize reasoning and retrieval coordination, enabling the model to perform coherent multi-step planning and adaptively invoke KG and external tools under incomplete knowledge conditions. We start with the definition of the KGQA task.

3.1 Task Formulation

Let D be a dataset containing question–answer pairs (q, a) , $\mathcal{G}$ a KG containing a set of triples $\{(e_i, r, e_j) \mid e_i, e_j \in \mathcal{E}; r \in \mathcal{R}\}$, where $\mathcal{E}$ is a set of entities and $\mathcal{R}$ is the set of relations names, and E an external search engine. The KGQA task requires the LLM to generate reasoning trajectories or to iteratively interact with $\mathcal{G}$ and E . Formally, for each question q , we generate a reasoning trajectory: $o = (\tau_1, \tau_2, \dots, \tau_T)$, where the t -th intermediate reasoning

step $\tau_t = (s_t, c_t)$ consists of an action $s_t \in \{\langle think \rangle, \langle plan \rangle, \langle relation_search \rangle, \langle neighbor_search \rangle, \langle web_search \rangle, \langle answer \rangle\}$ (cf. Figure 7) and its associated content c_t . The model is expected to repeatedly retrieve knowledge from $\mathcal{G}$ and E until reaching the final answer a for the question q .

3.2 CoT Fine-Tuning for Reasoning Activation

In the initial phase, we propose the CoT finetuning method using a structured reasoning dataset that contains step-by-step planning, reasoning, and retrieval processes. This phase is critical for activating the model’s planning-reasoning capabilities and establishing a robust cold start for subsequent RL. To construct our SFT dataset, as illustrated in Figure 2, we monitor and record the agent’s output throughout a multi-step interactive process with the KG and the external search engine. This approach transforms the complex, multi-step collaboration procedure into a CoT-style trajectory suitable for SFT. In addition, we use a two-stage process to filter the trajectory to get the high quality dataset. The Details regarding the data’s annotation and filtering processes are provided in [Appendix A.1](#). The training objective minimizes:

$$\mathcal{L}_{SFT} = - \sum_{t=1}^T \log \pi_{\theta}(y_t \mid q, y_{<t}) \quad (1)$$

where q is the question, y is the concatenated sequence of reasoning steps and the answer, π_{θ} is the model’s token distribution. The output model π_{CoT} serves as the initialization for the next stage [35], ensuring a robust foundation for RL.

3.3 RL-based Reasoning Enhancement

3.3.1 Rollout with Search Tools. Our approach follows an iterative framework where the LLM alternates between text generation and external search engine queries. Specifically, our method consists of three steps: i) Plan the sub-problem decomposition and tool invocation for complex reasoning in a specific order; ii) Retrieve corresponding answers from the knowledge graph based on the priority and logical functions of these subproblems; iii) Obtain the correct answers by searching external sources for information not available in the knowledge graph.

Plan the Steps. We adhere to the Cartesian principle of breaking down complex problems into distinct steps, while designing logical functions to assist the model in understanding the interrelationships between sub-problems and determining the appropriate timing for tool utilization. In Figure 2, the first two steps correspond to subproblems of higher priority, as they do not rely on answers from other subproblems. Additionally, we use logical functions to represent the parameters for invoking KG retrieval tool, which helps the model gain a clear understanding of the invocation process. Step 3 expresses the logical relationship between the two subproblems through the $inter(Ans_1, Ans_2, \dots, Ans_n)$ function (the intersection of the answers to subquestions), passes the returned results to Step 4, and obtains the final answer by invoking the KG retrieval tool. The planning steps are encapsulated between $\langle plan \rangle$ and $\langle /plan \rangle$.

Retrieve Facts from the KG. For each single steps obtained through planning, we need to sequentially select relevant triples from the KG. To address this, we design two interactive toolkits.

- **Relation Search Tool.** This tool retrieves a set of candidate relations for an entity in a given single-hop problem. The LLM encapsulates the entity and the hypothetical relation from the logical function between the relation search tokens $\langle relation_search \rangle$ and $\langle /relation_search \rangle$. By invoking the relation acquisition toolkit, the set of top 15 relations that are similar to the hypothetical relation is encapsulated between the tokens $\langle relation_information \rangle$ and $\langle /relation_information \rangle$.
- **Neighbor Search Tool.** This tool retrieves the tail entity corresponding to a given head entity-relation pair selected by LLMs. The LLM encapsulates the specified head entity and relation between the neighbor search tokens $\langle neighbor_search \rangle$ and $\langle /neighbor_search \rangle$. By invoking the neighbor acquisition toolkit, the answer is then encapsulated between the tokens $\langle neighbor_information \rangle$ and $\langle /neighbor_information \rangle$.

Given the obtained single-hop questions (e.g., “Identify the country that uses the Iranian Rail”), we first invoke the Relation Search Tool to retrieve a candidate set of relations associated with the initial (topic) entity. The LLM then reasons over this set and selects the most appropriate relation, for example, choosing “location.country.currency_used” to replace “used_in” in the retrieval function. This greedy strategy simplifies the reasoning process by selecting a locally optimal relation at each step, thereby reducing reliance on global contextual information. Next, the Neighbor Search Tool is called to obtain specific triples and their corresponding answers (e.g., “Iran”). If the LLM determines that the retrieved triples are insufficient to answer the question, or if no answer is found in the knowledge graph (i.e., when $\langle neighbor_information \rangle$ and $\langle /neighbor_information \rangle$ return “No information in the KG, please use web tool”), the system directly invokes the search engine. The answers to higher-priority (according to the order produced by the subquestion plan) single-hop questions can then be combined with those of lower-priority ones to generate new single-hop questions. By iteratively repeating this process, the system is able to derive answers to complex multi-hop questions. Notably, if the maximum iteration limit is reached without successfully generating an answer, the parametric knowledge of the LLM is used to provide a final response.

When LLMs determine that the knowledge retrieved through KG search tools is insufficient or implausible (such as when the KG lacks relevant information, i.e., the tags $\langle neighbor_information \rangle$ and $\langle /neighbor_information \rangle$ return “No information in KG, please use web tool.”), the model encapsulates the corresponding head entity and relation, for which no information is available, within the special search tokens $\langle web_search \rangle$ and $\langle /web_search \rangle$. Subsequently, a search engine is invoked, and the documents retrieved from the Bing website are enclosed within $\langle web_information \rangle$ and $\langle /web_information \rangle$. This mechanism not only enables the model to acquire the supplementary knowledge required to answer the query but also facilitates the identification of missing triples, which can later be incorporated into the KG to mitigate its incompleteness. In our experiments, we fixed the top- k value (e.g., 3) to control the number of retrieved documents.

3.3.2 Reward Design. To further optimize the above retrieval-planning process, we propose the GRPO [31] based RL method with the novel multi-reward design. Prior studies have predominantly

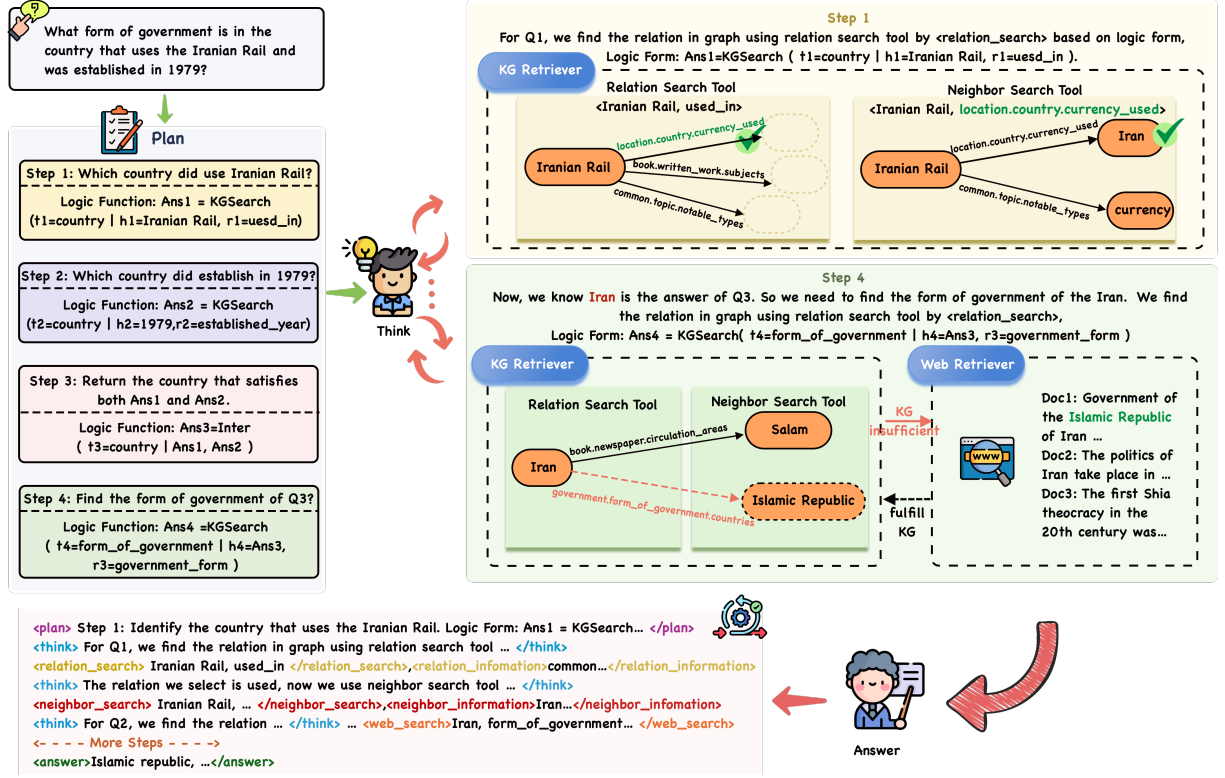


Figure 2: Graph-RFT Reasoning Trajectory for SFT dataset construction and rollout process for RL.

employed outcome-based rewards to guide LLMs in developing reasoning abilities and leveraging search engines throughout the reinforcement learning process. However, such supervision alone does not sufficiently capture diverse retrieval-oriented reward mechanisms and is inadequate for effectively guiding retrieval in the presence of incomplete KGs. To address this limitation, we incorporate two key components into our reward design: (i) an outcome-based reward, which directly evaluates the quality of the model’s generated answers; and (ii) a retrieval-specific reward, which encourages the model to discern when and how to extract relevant information from knowledge graphs or web documents.

Outcome-based Reward. Result-based reward are divided into format reward and answer reward. For format reward, we first define the correct format to ensure that the LLM adopts the predefined iterative workflow of "Think-Decompose-Retrieval-Search". The definitions of the model’s thinking process, tool invocation, and final answer output format are shown in Figure 7 in Appendix A.4. For answer rewards $r_{ans} \in [0, 1]$, we compare the predicted answer in <answer></answer> with the ground-truth answer, and use the F1-score between the two sets as the reward to measure its accuracy. The complete accuracy reward R_{acc} is defined as:

$$R_{acc} = \begin{cases} \max(0.1, r_{ans}), & \text{if format is correct,} \\ 0, & \text{if format is incorrect.} \end{cases} \quad (2)$$

$$r_{ans} = F1(p_{ans}, a) = \frac{2|p_{ans} \cap \{a\}|}{|p_{ans}| + |a|} \quad (3)$$

where p_{ans} is the predicted answer, and a is the ground truth answer from the (q, a) pair.

Retrieval-specific Reward. Inspired by AutoRefine [32], building on the result-based reward described above, we further introduce two retrieval rewards to encourage LLMs to learn how to extract relevant information from both the KG and web documents. Specifically, we introduce a graph retrieval reward R_{graph} and a document search reward R_{web} .

R_{graph} is measured based on the retrieval results contained in the <neighbor_information></neighbor_information> block. Specifically, we collect all graph retrieval results from the entire trajectory and concatenate them into a single text sequence:

$$R_{graph} = \mathbb{I}(\{a\} \cap o_{graph} = a) \quad (4)$$

$$o_{graph} = \bigcup \{c_t \mid (s_t, c_t) \in o \wedge s_t = \text{<neighbor_information>}\} \quad (5)$$

where $I(\cdot)$ is the indicator function, o_{graph} is the concatenation of all the KG retrieval information steps.

R_{web} is defined in a similar way to R_{graph} . We collect all document retrieval results encapsulated in the <web_information></web_information> blocks throughout the entire trajectory and concatenate them into a single text sequence.

$$R_{web} = \mathbb{I}(\{a\} \cap o_{web} = a) \quad (6)$$

$$o_{web} = \bigcup \{c_t \mid (s_t, c_t) \in o \wedge s_t = \text{<web_information>}\} \quad (7)$$

where $I(\cdot)$ is the indicator function, o_{graph} is the concatenation of all the web search information steps.

Table 2: Results of Graph-RFT across various datasets (represent the maximum hop), compared with the state-of-the-art (SOTA) in Supervised Learning (SL) and In-Context Learning (ICL) methods. The highest scores for ICL methods are highlighted in bold, while the second-best results are underlined. The Prior FT (Fine-tuned) SOTA includes the best-known results achieved through supervised learning. CKG denotes using complete knowledge graph and IKG denotes using incomplete KG (IKG-40%).

Method	Class	LLM	Multi-Hop KGQA						Single-Hop KGQA	
			CWQ (4-hop)		WebQSP (2-hop)		GrailQA (4-hop)		Simple Questions (1-hop)	
Without external knowledge										
IO prompt[33]	-	GPT-3.5-Turbo	37.6		63.3		29.4		20.0	
CoT[33]	-	GPT-3.5-Turbo	38.8		62.2		28.1		20.3	
SC[33]	-	GPT-3.5-Turbo	45.4		61.1		29.6		18.9	
			CKG	IKG	CKG	IKG	CKG	IKG	CKG	IKG
With external knowledge										
RoG [26]	SL	Qwen2.5-7B-base	63.7	53.6	84.2	75.3	-	-	-	-
ChatKBQA [24]	SL	Qwen2.5-7B-base	72.4	37.8	75.5	47.1	-	-	-	-
KB-BINDER[19]	ICL	Codex	-	-	50.7	38.4	-	-	-	-
StructGPT[13]	ICL	GPT-3.5-Turbo	-	-	76.4	60.1	-	-	-	-
ToG/ToG-R[33]	ICL	GPT-3.5-Turbo	47.2	37.9	76.9	63.4	-	-	-	-
ToG/ToG-R[33]	ICL	GPT-4	71.0	56.1	80.3	71.8	-	-	-	-
PoG[36]	ICL	GPT-4	76.8	62.6	96.7	76.4	79.2	64.1	80.2	<u>59.5</u>
GoG[44]	ICL	GPT-3.5-Turbo	55.7	44.3	78.7	66.6	71.7	62.4	54.3	43.9
GoG[44]	ICL	GPT-4	75.2	60.4	84.4	80.3	76.8	69.4	68.9	56.4
Graph-RFT-instruct	RL	Qwen2.5-7B-instruct	<u>78.4</u>	<u>64.8</u>	87.3	<u>82.7</u>	<u>82.1</u>	<u>71.8</u>	73.8	58.7
Graph-RFT-base	RL	Qwen2.5-7B-base	80.7	67.2	<u>90.6</u>	86.3	84.6	73.3	<u>76.9</u>	62.4

However, such multi-turn reinforcement learning is unstable. To prevent the model from bypassing graph retrieval and directly turning to document search in cases where the correct result could have been obtained through graph retrieval, we introduce a corresponding penalty for this behavior to help LLMs know when to use different retrieval tools. The overall reward $\mathcal{R}_{\text{over}}$ can be formally written as:

$$\mathcal{R}_{\text{over}} = \begin{cases} \mathcal{R}_{\text{acc}}, & \text{if } \mathcal{R}_{\text{acc}} > 0, \\ 0.1, & \text{if } \mathcal{R}_{\text{acc}} = 0 \text{ \& } \mathcal{R}_{\text{graph}} > 0 \text{ OR } \mathcal{R}_{\text{acc}} = 0 \text{ \& } \mathcal{R}_{\text{web}} > 0, \\ -0.1, & \text{if } \mathcal{R}_{\text{web}} > 0 \text{ when CKG OR } \mathcal{R}_{\text{web}} = 0 \text{ when IKG} \\ 0, & \text{if } \mathcal{R}_{\text{acc}} = \mathcal{R}_{\text{web}} = \mathcal{R}_{\text{graph}} = 0. \end{cases} \quad (8)$$

Training Objective. By integrating the overall reward with the GRPO training objective [31], our proposed learning objective enables dynamic interaction with both the knowledge graph and the search engine during optimization [16], thereby facilitating effective LLM search training. The objective is formalized as:

$$\max_{\pi_{\theta}} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(\cdot|x; \mathcal{G}, \mathcal{R})} [r_{\phi}(x, y)] - \beta \mathbb{D}_{\text{KL}} [\pi_{\theta}(y|x; \mathcal{G}, \mathcal{R}) \| \pi_{\text{ref}}(y|x; \mathcal{G}, \mathcal{R})] \quad (9)$$

where π_{θ} denotes the trainable policy model, π_{ref} is a fixed reference model, r_{ϕ} represents the overall reward function, and $\mathbb{D}_{\text{KL}}$ denotes the KL divergence. Here, x is sampled from the dataset $\mathcal{D}$, and y denotes the output sequence interleaving reasoning steps with KG and search engine retrievals. Since the retrieved triples and documents are not generated by the policy model, we mask the retrieval results during loss calculation to prevent the training policy from being biased [10].

4 Experiments

To evaluate the effectiveness of Graph-RFT, we aim to explore the following four research questions (RQs): (1) **RQ1**: How does Graph-RFT perform against SOTA baselines on complex reasoning datasets? (2) **RQ2**: What are the contributions of each Graph-RFT module to overall performance? (3) **RQ3**: Can Graph-RFT effectively bridge information gaps via retrieval under varying KG incompleteness? (4) **RQ4**: What are the error types of Graph-RFT?

► **Datasets.** We evaluate Graph-RFT on four KGQA datasets, including three multi-hop datasets: CWQ [34], WebQSP [46], GrailQA [8] and one single-hop dataset: SimpleQuestions [28]. We consider two scenarios: Complete Knowledge Graph (CKG) and Incomplete Knowledge Graph (IKG). For CKG, we directly use the datasets provided by GoG [44]. For IKG, since GoG only provides CWQ and WebQSP, we construct IKG versions of GrailQA and SimpleQuestions by sampling 3,000 multi-hop questions from GrailQA and 3,000 questions from SimpleQuestions. We further generate four IKGs with varying completeness levels IKG-20%, IKG-40%, IKG-60% for example, IKG-40% removes 40% of the critical triples for each question and all relations between the corresponding entity pairs. Freebase [4] serves as the background KG for all datasets. The impact of using alternative knowledge graphs is discussed in [Appendix A.5](#).

► **Evaluation Metrics.** Following prior work [3, 13, 20, 33], we adopt exact match accuracy (Hits@1) as the evaluation metric for all datasets. Detailed descriptions of the baselines and implementation settings are provided in [Appendix A.2](#) and [Appendix A.3](#).

4.1 Main Results (RQ1)

Table 2 presents our main results on four KGQA datasets. We can observe that Graph-RFT consistently achieves SOTA performance across all settings in IKG, even when built upon smaller backbones such as Qwen2.5-7B-base, outperforming GPT-4-based methods.

► **CKG Setting.** Under the CKG setting, where all relevant triples are available, Graph-RFT achieves the strongest overall performance across both multi-hop and single-hop benchmarks. The results reveal that our two-stage RFT which combines CoT-based reasoning activation and plan-retrieval RL—provides consistent advantages over both prompt-only ICL methods and supervised learning baselines. ICL approaches such as PoG and GoG rely purely on prompt engineering and lack any fine-tuning or reinforcement optimization. Although they perform competitively on simpler datasets such as WebQSP and SimpleQuestions, they struggle to generalize across long reasoning chains. In contrast, Graph-RFT-base (Qwen-2.5-7B) achieves substantial gains on complex benchmarks like CWQ (80.7 vs 76.8/75.2) and GrailQA (84.6 vs 79.2/76.8), where multi-step logical dependencies are required. These improvements highlight how explicit CoT supervision enables structured planning, and reinforcement optimization refines the timing and sequencing of retrieval actions. Relative to RoG and ChatKBQA, which are purely supervised fine-tuned models, Graph-RFT demonstrates marked improvements (e.g., CWQ 80.7 vs 72.4/63.7, WebQSP 90.6 vs 84.2/75.5). This suggests that reinforcement learning contributes beyond imitation: by rewarding accurate retrieval scheduling and penalizing inefficient queries, the model learns to maintain global reasoning consistency and to balance exploration between relation-search and neighbor-search tools.

Table 3: Ablation study for the Graph-RFT-base in rollout framework.

	PS	KR	WR	Multi-Hop KGQA			Single-Hop KGQA
				CWQ	WebQSP	GrailQA	Simple Questions
Variants	-	✓	-	49.4	69.7	58.9	53.8
	-	✓	✓	55.3	74.2	63.5	<u>61.7</u>
	✓	✓	-	<u>62.6</u>	<u>83.4</u>	<u>69.1</u>	57.3
Graph-RFT-base	✓	✓	✓	67.2	86.3	73.3	62.4

Table 4: Ablation analysis on Graph-RFT-base SFT and multi-reward design.

Method	Multi-Hop KGQA			Single-Hop KGQA
	CWQ	WebQSP	GrailQA	Simple Questions
Graph-RFT-base w/o SFT	46.4	74.2	56.4	54.7
Graph-RFT-base w/o R_{web}	66.3	83.5	72.1	60.8
Graph-RFT-base w/o R_{graph}	65.8	82.7	71.6	61.6
Graph-RFT-base	67.2	86.3	73.3	62.4

► **IKG Setting.** When the knowledge graph is incomplete, the benefits of Graph-RFT become even more pronounced. Prompt-based ICL approaches such as PoG and GoG (GPT-4) achieve competitive accuracy under complete KGs but degrade significantly once KG coverage is reduced. For example, GoG’s performance drops 7-15 points across CWQ (75.2→60.4) and GrailQA (76.8→69.4), while

PoG falls from 76.8→62.6 on CWQ. These methods rely on static prompt patterns and local reasoning, lacking mechanisms to assess KG sufficiency or to seek supplementary evidence. In contrast, Graph-RFT-base (Qwen2.5-7B) remains stable across both settings, outperforms among ICL methods. This resilience arises from our two-stage RFT: CoT fine-tuning teaches explicit question decomposition, and reinforcement optimization learns when and how to invoke external retrievals. SL methods such as RoG and ChatKBQA also decline sharply under IKG (e.g., CWQ: 63.7→53.6; ChatKBQA: 72.4→37.8), as their fixed fine-tuning lacks interactivity with retrieval tools. In contrast, Graph-RFT maintains high accuracy (e.g., CWQ 67.2, WebQSP 86.3), confirming that reinforcement-based retrieval scheduling effectively compensates for missing triples through controlled web augmentation. Moreover, Graph-RFT with Qwen2.5-7b-base outperforms methods without external knowledge (e.g., IO, CoT, SC prompting), demonstrating that KGs play a crucial role in reasoning tasks.

4.2 Ablation Studies (RQ2)

We perform ablation studies to analyze the contributions of Graph-RFT’s key components. Specifically, we examine the effects of the rollout framework, SFT and the multi-reward mechanism in the RL stage.

► **Exploration on Rollout Framework.** We analyze the contributions of the Planning Steps (PS), KG Retrieval (KR), and Web Retrieval (WR) modules within the rollout framework to Graph-RFT’s performance under the IKG-40% setting. Figure 3 presents model variants obtained by ablating these components, revealing that removing any single module substantially reduces performance. Comparing PS+KR, KR+WR, and KR alone, we observe that PS+KR achieves superior performance on multi-hop datasets, indicating that global planning—including question decomposition with logical operators and tool invocation—provides greater benefit than merely supplementing missing knowledge, thereby validating the effectiveness of the PS module. In contrast, on the single-hop dataset, KR+WR outperforms PS+KR, suggesting that leveraging external unstructured knowledge becomes more critical than global planning for simpler queries, which demonstrates Graph-RFT’s adaptability across tasks of varying complexity. Furthermore, the comparison between KR+WR and KR alone confirms that integrating structured knowledge from the KG with unstructured knowledge from web documents produces substantial improvements. Collectively, these results indicate that each module contributes uniquely, and their synergistic combination significantly enhances the model’s capability for complex reasoning tasks.

► **Exploration on SFT and Multi-reward Design.** We further examine the effect of the SFT stage and the multi-reward mechanism on Graph-RFT’s performance under the IKG-40% setting. Comparing Graph-RFT-base w/o SFT with Graph-RFT-base shows a marked performance drop across all datasets, underscoring the crucial role of SFT in strengthening model capability. Moreover, removing either the web retrieval reward or the graph retrieval reward consistently degrades performance, demonstrating that both rewards effectively guide the LLM to retrieve more semantically relevant triples and external documents. As shown in Figure 3, when the model’s web search frequency approaches the optimal

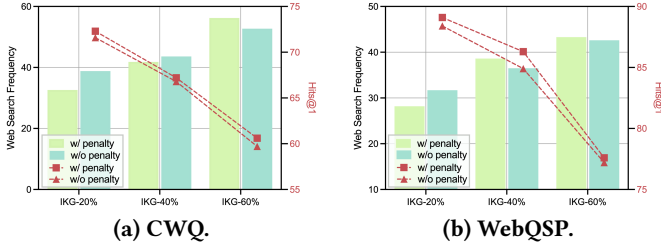


Figure 3: Ratio of web search operation in different KG settings under rewards with penalty or not.

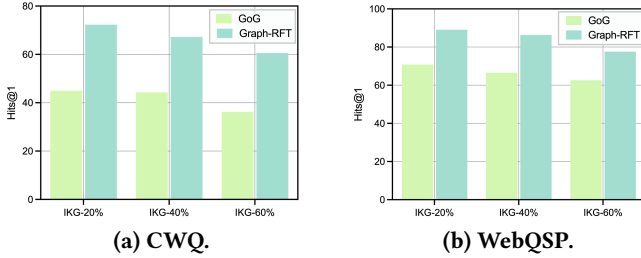


Figure 4: Performance of GoG and our model under different KG settings.

level (e.g., 20% in the IKG-20% setting), performance on CWQ and WebQSP improves. This indicates that Graph-RFT can adaptively regulate web search frequency based on the sufficiency of KG information, reducing the risk of noisy or irrelevant content and thereby enhancing overall effectiveness.

4.3 Different KG Incompleteness (RQ3)

The key capabilities of our model lies in performing global planning on complex problems to leverage the knowledge within the KG and identifying knowledge gaps beyond the KG through web search. We examine the performance of Graph-RFT under varying degrees of KG incompleteness and record its external search frequency, as shown in Figure 3. To evaluate the robustness of different methods under KG incompleteness, we compare GoG and Graph-RFT across KGs with different completeness levels, as illustrated in Figure 4.

► **Impact of Rewards with or without Penalty.** From Figure 3, Graph-RFT increases the frequency of external searches as KG completeness decreases. This behavior reflects the model’s ability to proactively compensate for missing knowledge required for reasoning through web retrieval. Moreover, the model’s search strategy varies notably with question complexity: compared to WebQSP (maximum 2-hop), Graph-RFT conducts significantly more external searches when tackling the more complex multi-hop questions in CWQ (maximum 4-hop). This demonstrates that the model can dynamically adjust its web search frequency according to question complexity, effectively aligning knowledge acquisition with the reasoning demands of the task.

► **Impact of Different KG Incompleteness.** From Figure 4, we observe that Graph-RFT outperforms GoG across all degrees of

incompleteness. This improvement arises because GoG is primarily confined to local decision-making, lacking the global planning needed for effective question understanding and tool invocation in complex problem decomposition. In contrast, when KG triples are missing, GoG relies solely on the LLM’s internal knowledge to generate answers, whereas our model retrieves relevant external documents through web search, thereby supplementing incomplete KG information and improving answer accuracy. Notably, the improvement on CWQ is much more pronounced than on WebQSP, validating Graph-RFT’s strong planning ability and logical decomposition mechanism for tackling complex reasoning tasks.

4.4 Error Type Analysis (RQ4)

To analyze Graph-RFT’s limitations, we randomly selected 50 failure cases from CWQ and WebQSP under both CKG and IKG settings. These cases are categorized into five types: (1) invalid actions, (2) relation selection errors, (3) neighbor selection errors, (4) reasoning errors, and (5) decomposition errors, with their distribution shown in Figure 6. We can obtain the following findings: (i): reasoning errors—where the LLM follows a correct reasoning process but produces incorrect answers—represent the largest proportion in both datasets and are more prevalent in IKG than in CKG, likely due to answer aliasing and discrepancies between retrieved document answers and reference answers. (ii): neighbor selection errors manifest in two scenarios: on the one hand, when relation selection is correct, neighbor errors are relatively rare in CKG, indicating that the model primarily struggles with relation selection; on the other hand, when the correct relation is not identified, as in IKG, the model often fails to determine whether entities are sufficient for problem-solving, resulting in incorrect neighbor selection. (iii): decomposition errors, arising from flawed planning, occur more frequently on complex problems such as CWQ, whereas invalid action errors (e.g., invoking undefined tools) are effectively managed by the model. The case study is in [Appendix A.6](#).

5 Conclusion

We introduced Graph-RFT, a two-stage reinforcement fine-tuning framework for complex reasoning over incomplete knowledge graphs. Graph-RFT unifies structured planning and adaptive retrieval within a single learning paradigm, enabling LLMs to reason coherently and retrieve information dynamically across KGs and web sources. In the first stage, a CoT fine-tuning process with a customized plan–retrieval dataset activates the model’s planning and reasoning capabilities while mitigating the GRPO cold-start problem. In the second stage, a plan–retrieval guided reinforcement learning procedure with a multi-reward design optimizes coverage-aware retrieval scheduling and multi-step logical consistency. Extensive experiments across multiple KGQA benchmarks demonstrate that Graph-RFT consistently outperforms strong baselines, achieving superior accuracy even with smaller LLM backbones. In the future, we will explore enhancing relation filtering performance from KGs and the capability of decomposing problems when dealing with complex reasoning tasks.

Acknowledgments

We thank all anonymous reviewers and area chairs for their comments. This work is supported by the National Natural Science Foundation of China (U23A20316), the CCF-Tencent Rhino-Bird Open Research Fund (CCF-Tencent RAGR20250115), and the computing power resources from Wuhan Dongxihu District Intelligent Computing Center.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Farah Atif, Ola El Khatib, and Djellel Difallah. 2023. Beamqa: Multi-hop knowledge graph question answering with sequence-to-sequence prediction and beam search. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 781–790.
- [3] Jinheon Baek, Alham Fikri Aji, and Amir Saffari. 2023. Knowledge-augmented language model prompting for zero-shot knowledge graph question answering. *arXiv preprint arXiv:2306.04136* (2023).
- [4] Kurt Bollacker, Colin Evans, Praveen Paritosh, Tim Sturge, and Jamie Taylor. 2008. Freebase: a collaboratively created graph database for structuring human knowledge. In *Proceedings of the 2008 ACM SIGMOD international conference on Management of data*. 1247–1250.
- [5] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457* (2018).
- [6] René Descartes. 1901. *Discourse on method*. Aladdin Book Company.
- [7] Tengfei Feng and Liang He. 2025. Rgr-kbqa: Generating logical forms for question answering using knowledge-graph-enhanced large language model. In *Proceedings of the 31st International Conference on Computational Linguistics*. 3057–3070.
- [8] Yu Gu, Sue Kase, Michelle Vanni, Brian Sadler, Percy Liang, Xifeng Yan, and Yu Su. 2021. Beyond iid: three levels of generalization for question answering on knowledge bases. In *Proceedings of the web conference 2021*. 3477–3488.
- [9] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
- [10] Chuzhan Hao, Wenfeng Feng, Yuewei Zhang, and Hao Wang. 2025. DynaSearcher: Dynamic Knowledge Graph Augmented Search Agent via Multi-Reward Reinforcement Learning. *arXiv preprint arXiv:2507.17365* (2025).
- [11] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300* (2020).
- [12] Sen Hu, Lei Zou, Jeffrey Xu Yu, Haixun Wang, and Dongyan Zhao. 2017. Answering natural language questions by subgraph matching over knowledge graphs. *IEEE Transactions on Knowledge and Data Engineering* 30, 5 (2017), 824–837.
- [13] Jinhao Jiang, Kun Zhou, Zican Dong, Keming Ye, Wayne Xin Zhao, and Ji-Rong Wen. 2023. Structgpt: A general framework for large language model to reason over structured data. *arXiv preprint arXiv:2305.09645* (2023).
- [14] Jinhao Jiang, Kun Zhou, Wayne Xin Zhao, Yang Song, Chen Zhu, Hengshu Zhu, and Ji-Rong Wen. 2024. Kg-agent: An efficient autonomous agent framework for complex reasoning over knowledge graph. *arXiv preprint arXiv:2402.11163* (2024).
- [15] Jinhao Jiang, Kun Zhou, Wayne Xin Zhao, and Ji-Rong Wen. 2022. Unikgqa: Unified retrieval and reasoning for solving multi-hop question answering over knowledge graph. *arXiv preprint arXiv:2212.00959* (2022).
- [16] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Serkan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. 2025. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516* (2025).
- [17] Hanieh Khorashadizadeh, Fatima Zahra Amara, Morteza Ezzabady, Frédéric Ieng, Sanju Tiwari, Nandana Mihindukulasooriya, Jinghua Groppe, Soror Sahri, Farah Benamara, and Sven Groppe. 2024. Research trends for the interplay between large language models and knowledge graphs. *arXiv preprint arXiv:2406.08223* (2024).
- [18] Yunshi Lan and Jing Jiang. 2020. Query graph generation for answering multi-hop complex questions from knowledge bases. Association for Computational Linguistics.
- [19] Tianle Li, Xueguang Ma, Alex Zhuang, Yu Gu, Yu Su, and Wenhui Chen. 2023. Few-shot in-context learning for knowledge base question answering. *arXiv preprint arXiv:2305.01750* (2023).
- [20] Xingxuan Li, Ruochen Zhao, Yew Ken Chia, Bosheng Ding, Lidong Bing, Shafiq Joty, and Soujanya Poria. 2023. Chain of knowledge: A framework for grounding large language models with structured knowledge bases. *arXiv preprint arXiv:2305.13269* 3 (2023).
- [21] Ben Liu, Jihai Zhang, Fangquan Lin, Cheng Yang, Min Peng, and Wotao Yin. 2025. Symagent: A neural-symbolic self-learning agent framework for complex reasoning over knowledge graphs. In *Proceedings of the ACM on Web Conference 2025*. 98–108.
- [22] Ziyu Liu, Zeyi Sun, Yuhang Zang, Xiaoyi Dong, Yuhang Cao, Haodong Duan, Dahua Lin, and Jiaqi Wang. 2025. Visual-rft: Visual reinforcement fine-tuning. *arXiv preprint arXiv:2503.01785* (2025).
- [23] Haoran Luo, Guanting Chen, Qika Lin, Yikai Guo, Fangzhi Xu, Zemin Kuang, Meina Song, Xiaobao Wu, Yifan Zhu, Luu Anh Tuan, et al. 2025. Graph-r1: Towards agentic graphrag framework via end-to-end reinforcement learning. *arXiv preprint arXiv:2507.21892* (2025).
- [24] Haoran Luo, Zichen Tang, Shiyao Peng, Yikai Guo, Wentai Zhang, Chenghao Ma, Guanting Dong, Meina Song, Wei Lin, Yifan Zhu, et al. 2023. Chatkbqa: A generate-then-retrieve framework for knowledge base question answering with fine-tuned large language models. *arXiv preprint arXiv:2310.08975* (2023).
- [25] Linhao Luo, Jiaxin Ju, Bo Xiong, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan. 2025. Chatrule: Mining logical rules with large language models for knowledge graph reasoning. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer, 314–325.
- [26] Linhao Luo, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan. 2023. Reasoning on graphs: Faithful and interpretable large language model reasoning. *arXiv preprint arXiv:2310.01061* (2023).
- [27] Jie Ma, Zhitao Gao, Qi Chai, Wangchun Sun, Pinghui Wang, Hongbin Pei, Jing Tao, Lingyun Song, Jun Liu, Chen Zhang, et al. 2025. Debate on graph: a flexible and reliable reasoning framework for large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 24768–24776.
- [28] M Petrochuk and L Zettlemoyer. [n. d.]. Simplequestions nearly solved: A new upperbound and baseline approach. *arXiv* 2018. *arXiv preprint arXiv:1804.08798* ([n. d.]).
- [29] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research* 21, 140 (2020), 1–67.
- [30] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems* 36 (2023), 68539–68551.
- [31] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300* (2024).
- [32] Yaorui Shi, Sihang Li, Chang Wu, Zhiyuan Liu, Junfeng Fang, Hengxing Cai, An Zhang, and Xiang Wang. 2025. Search and Refine During Think: Autonomous Retrieval-Augmented Reasoning of LLMs. *arXiv preprint arXiv:2505.11277* (2025).
- [33] Jiashuo Sun, Chengjin Xu, Luminyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M Ni, Heung-Yeung Shum, and Jian Guo. 2023. Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. *arXiv preprint arXiv:2307.07697* (2023).
- [34] Alon Talmor and Jonathan Berant. 2018. The web as a knowledge-base for answering complex questions. *arXiv preprint arXiv:1803.06643* (2018).
- [35] Huajie Tan, Yuheng Ji, Xiaoshuai Hao, Xiansheng Chen, Pengwei Wang, Zhongyuan Wang, and Shanghang Zhang. 2025. Reason-rft: Reinforcement fine-tuning for visual reasoning of vision language models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- [36] Xingyu Tan, Xiaoyang Wang, Qing Liu, Xiwei Xu, Xin Yuan, and Wenjie Zhang. 2025. Paths-over-graph: Knowledge graph empowered large language model reasoning. In *Proceedings of the ACM on Web Conference 2025*. 3505–3522.
- [37] Qwen Team et al. 2024. Qwen2 technical report. *arXiv preprint arXiv:2407.10671* 2 (2024), 3.
- [38] Benyou Wang, Qianqian Xie, Jiahuan Pei, Zhihong Chen, Prayag Tiwari, Zhao Li, and Jie Fu. 2023. Pre-trained language models in biomedical domain: A systematic survey. *Comput. Surveys* 56, 3 (2023), 1–52.
- [39] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [40] Qianqian Xie, Qingyu Chen, Aokun Chen, Cheng Peng, Yan Hu, Fongci Lin, Xueqing Peng, Jimin Huang, Jeffrey Zhang, Vipina Keloth, et al. 2025. Medical foundation large language models for comprehensive text analysis and beyond. *npj Digital Medicine* 8, 1 (2025), 141.
- [41] Qianqian Xie, Weiguang Han, Zhengyu Chen, Ruoyu Xiang, Xiao Zhang, Yueru He, Mengxi Xiao, Dong Li, Yongfu Dai, Duanyu Feng, et al. 2024. Finben: A holistic financial benchmark for large language models. *Advances in Neural Information Processing Systems* 37 (2024), 95716–95743.

- [42] Qianqian Xie, Weiguang Han, Xiao Zhang, Yanzhao Lai, Min Peng, Alejandro Lopez-Lira, and Jimin Huang. 2023. Pixiu: A comprehensive benchmark, instruction dataset and large language model for finance. *Advances in Neural Information Processing Systems* 36 (2023), 33469–33484.
- [43] Chenyan Xiong, Russell Power, and Jamie Callan. 2017. Explicit semantic ranking for academic search via knowledge graph embedding. In *Proceedings of the 26th international conference on world wide web*. 1271–1279.
- [44] Yao Xu, Shizhu He, Jiabei Chen, Zihao Wang, Yangqiu Song, Hanghang Tong, Guang Liu, Kang Liu, and Jun Zhao. 2024. Generate-on-graph: Treat llm as both agent and kg in incomplete knowledge graph question answering. *arXiv preprint arXiv:2404.14741* (2024).
- [45] Shangzi Xue, Zhenya Huang, Jiayu Liu, Xin Lin, Yuting Ning, Binbin Jin, Xin Li, and Qi Liu. 2024. Decompose, analyze and rethink: Solving intricate problems with human-like reasoning cycle. *Advances in Neural Information Processing Systems* 37 (2024), 357–385.
- [46] Wen-tau Yih, Matthew Richardson, Christopher Meek, Ming-Wei Chang, and Jina Suh. 2016. The value of semantic parse labeling for knowledge base question answering. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. 201–206.
- [47] Jingying Zeng, Richard Huang, Waleed Malik, Langxuan Yin, Bojan Babic, Danny Shacham, Xiao Yan, Jaewon Yang, and Qi He. 2024. Large language models for social networks: Applications, challenges, and solutions. *arXiv preprint arXiv:2401.02575* (2024).
- [48] Zhiqiang Zhang, Liqiang Wen, and Wen Zhao. 2025. Rule-KBQA: rule-guided reasoning for complex knowledge base question answering with large language models. In *Proceedings of the 31st International Conference on Computational Linguistics*. 8399–8417.
- [49] Qian Zhao, Hao Qian, Ziqi Liu, Gong-Duo Zhang, and Lihong Gu. 2024. Breaking the barrier: utilizing large language models for industrial recommendation systems through an inferential knowledge graph. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 5086–5093.

A APPENDIX

A.1 SFT Dataset Construction

We construct a Long CoT dataset comprising 4862 samples by prompting QwQ-32B with KG and Web retrieval tools. For quality filtering, we formulate two stages filtering processes:

► **Format Check.** COT trajectories must not contain any labels except the defined ones, and the `<plan></plan>` can only be invoked once.

► **Correctness Check.**

- **Answer Correctness:** The answer contained within the label must be correct; otherwise, the trajectory will be filtered out.
- **Retrieval Correctness:** When the knowledge graph is complete: The COT must not contain the `<web_search>`, and the `<neighbor_information>` must include the correct answer. If not, the trajectory will be filtered out. When the knowledge graph is incomplete: The COT contains the `<web_search>` and the `<web_information>` must includes the correct answer. If not, the trajectory will be filtered out.
- **Plan Correctness:** GPT-4 is used to determine whether the decomposition and planning in the `<plan>` section are reasonable. A score of 1 is assigned for reasonable planning, and 0 for unreasonable planning. Trajectories with a score of 0 are filtered out.

A.2 Baselines

We assess the performance of Graph-RFT, leveraging Qwen2.5-7B as the backbone model. The baselines we compare can be divided into three groups: (1) LLM-only methods, including standard In-Context (IO) prompting, Chain-of-Thought (CoT) prompting, and Self-Consistency (SC) prompting. All comparisons utilize six in-content examples and incorporate "step-by-step" reasoning chains. (2) Semantic Parsing methods, including KB-BINDER[19], ChatKBQA[24]. (3) Retrieval Augmented methods, including StructGPT[13], RoG[26], ToG[33], PoG[36], GoG[44].

A.3 Implementation Details

We use Qwen2.5-7B-instruct and Qwen2.5-7B-base as backbones [37] to obtain Graph-RFT-instruct and Graph-RFT-base, respectively, using PyTorch on 8 NVIDIA A800 (80GB) GPUs. In the SFT stage, we leverage LLaMA-Factory framework for efficient LLM fine-tuning. We employ a batchsize of 256 for 2 epochs with a learning rate of $1.4e-5$ and AdamW optimizer with cosine decay. In the RL stage, we adopt the Verl framework with the batch size of 128 while the mini-batch size of 32, a rollout number of 8.

A.4 Prompt for Graph-RFT

In this section, we present the prompt template. The prompt comprises core components that guide the reasoning and interaction process with the knowledge graph. As illustrated in Figure 7, we adopt a structured interaction format, which utilizes the `<plan>`, `<relation_search>`, `<neighbor_search>`, `<web_search>`, and `<answer>`. To facilitate complex reasoning, Graph-RFT first formulates a global plan. This plan outlines how to decompose the original question into manageable sub-problems using the custom-designed logic functions. Following the planning phase, each sub-problem is retrieved step-by-step from the KGs, with the invocation of dedicated KG retrieval tools. Subsequently, Graph-RFT employs a clearly defined set of tools to interact with both the knowledge graph and external documents. Through this interaction, it performs multi-step reasoning and ultimately derives the final answer. By integrating structured knowledge from the incomplete KG with supplementary information from external sources, this systematic approach enables Graph-RFT to effectively address complex problems.

A.5 Different Knowledge Bases for Graph-RFT

As shown in Table 5, Graph-RFT achieves significant improvements with different source KGs on CWQ and WebQSP in IKG-40%, compared to ToG. On the other hand, different source KGs might have different effects on the performance of ToG. Notably, Freebase brings more significant improvements on CWQ and WebQSP than Wikidata, since both datasets are constructed upon Freebase.

Table 5: Performances of Graph-RFT using different source KGs on CWQ and WebQSP in IKG-40%.

Method	CWQ	WebQSP
w/Freebase (ToG)	34.9	58.4
w/WikiData (ToG)	32.6	54.7
w/Freebase (Graph-RFT)	67.2	86.3
w/WikiData (Graph-RFT)	61.5	77.8

A.6 Case Study

To demonstrate Graph-RFT’s real-world operation, we provide a detailed case study that breaks down its reasoning process when

addressing complex questions. As shown in Figure 8, this example focuses on a team-related problem, illustrating how Graph-RFT systematically processes the question by leveraging both the knowledge graph and web resources.

Prompt Template of Graph-RFT

Given the question, you must conduct reasoning step by step with a priority logical relationship in accordance with Descartes' Principles inside `<think>` and `</think>`, which enables the system to start from the most basic sub-problems that do not rely on any prior information. As each sub-problem is solved, its answer will serve as the background basis for subsequent retrieval, thereby guiding the handling of more dependent sub-problems step by step. After thinking the whole reasoning process, you must give the plan process inside `<plan>` and `</plan>`, which encompass each step's problem and the logic functions of them. The functions are as follows:

Sub-problem Search Function:

`Ans=SearchKG ( t=target_type | h = topic_entity, r = relation_type of h and t)` describe given the topic entity and relation of the topic entity and target type in sub-problem, please use KG retrieval tool to find the answer. If the sub-problem need the predefined sub-problem's answer, we can use `Ans2=SearchKG ( t=target_type | h = Ans1, r = relation_type of h and t)` to plan the sequential relationship of subproblems.

Sub-problem Logic Functions:

`Intersect (sub-problem1, sub-problem2...)` describe the intersection of the answer of the sub-problems.

`Union (sub-problem1, sub-problem2...)` describe the union of the answer of the sub-problems.

`Negation (sub-problem1, sub-problem2...)` represents remove all elements from the sub-problem1 (main set) that match sub-problem2..., and return a new set that does not contain these.

After planning the whole process, you start with the first step. If you need to search in KG when `SearchKG()` exist, You must first call `<relation_search> h, r </relation_search>`, and it will return top15 relevant relation for you to select based on sub-problem within `<relation_information>` and `</relation_information>`. If there is no relevant relation, you can call a web search engine by `<web_search>h, r</web_search>`.

After getting the relation, you must second call a neighbor search engine by `<neighbor_search> h, r </neighbor_search>` and it will search the neighbors of the entity with the specified relation in the KG between `<neighbor_information>` and `</neighbor_information>`.

If there is no neighbor information, you can call a web search engine by `<web_search>h, r</web_search>` and it will return the top searched results between `<web_information>` and `</web_information>`.

You can search based on plan process, If you find no further external knowledge needed, you can directly provide the answer inside `<answer>` and `</answer>`

Figure 7: Prompt template of our Graph-RFT.

Case Study

Question: With a mascot named Rocky, which team does Nate Robinson play for?

`<think>`I need to know that which team has the mascot named Rocky first. Then I need to know which team Nate Robinson plays for. The final answer should satisfy both two answers. `</think>`
`<plan>` Step 1: Which team has a mascot named Rocky?

`Ans1 = SearchKG( t1=team | h1=Rocky, r1=has_mascot)`

Step 2: Which team does Nate Robinson play for?

`Ans2 = SearchKG( t2=team | h2=Nate Robinson, r2= play_for)`

Step 3: Return the answers that satisfies both Ans1 and Ans2.

`Ans3 = Inter(Ans1, Ans2) </plan>`

`<think>` Follow the plan part. For step 1, I need to find the answer of "Which team has a mascot named Rocky" in KG. So I should know the surrounding relation of Rocky, which is similar to `has_mascot`. `</think>`

`<relation_search>` Rocky, has_mascot `</relation_search>`

`<relation_information>`The relations are ['common.topic.notable_for', 'common.topic.article', 'common.topic.notable_types', 'sports.sports_team.team_mascot']. You should select one most potential relation. `</relation_information>`

`<think>`From the relation, the most potential one to answer the question is

`[sports.sports_team.team_mascot]`. I will search the neighbor of entity Rocky with this relation. `</think>`

`<neighbor_search>`Rocky, sports.sports_team.team_mascot`</neighbor_search>`

`<neighbor_information>` Denver Nuggets `</neighbor_information>`

`<think>` I know Denver Nuggets is the answer of question "Which team has a mascot named Rocky?". Now, for step 2, I need to find the answer of "Which team does Nate Robinson play for?" in KG. So I should know the surrounding relation of Nate Robinson, which is similar to `play_for`. `</think>`

`<relation_search>` Nate Robinson, play_for `</relation_search>`

`<relation_information>`The relations are ['sports.pro_athlete.teams',

'basketball.basketball_player.player_statistics',...

'sports.sports_team_roster.player']. You should select one most potential relation. `</relation_information>`

`<think>`From the relation, the most potential one to answer the question is

`[sports.pro_athlete.teams]`. I will search the neighbor of entity Rocky with this relation. `</think>`

`<neighbor_search>`Nate Robinson, sports.pro_athlete.teams`</neighbor_search>`

`<neighbor_information>` No information found under this relation in the knowledge graph. You can keep searching on graph or go to invoke search engine to retrieve relevant documents. You can also just answer if you think you can based on the information available and your knowledge. `</neighbor_information>`

`<think>`Since there is no information under the relation 'sports.pro_athlete.teams' in the knowledge graph, I will search for the entity Nate Robinson in website to find out the team he played for. `</think>`

`<web_search>`Nate Robinson, play_for`</web_search>`

`<web_information>` "Nate Robinson": Nate Robinson Nathaniel Cornelius Robinson (born May 31, 1984) is an American professional basketball player for Homenetmen Beirut of the Lebanese Basketball League. Born in Seattle, Robinson played college basketball for the University of Washington in Seattle and was the 21st pick in the 2005 NBA draft. The 5'9" point guard has also played for the New York Knicks, Boston Celtics, Oklahoma City Thunder, Golden State Warriors, Chicago Bulls, and Denver Nuggets. Robinson is the NBA's first three-time slam dunk champion. In 2018 he plays BIG3 basketball for Tri-State and AFL flag football for Holdat concurrently, until Holdat was eliminated. `</web_information>`

`<think>`Based on the information from website, I know that Robinson played for the New York Knicks, Boston Celtics, Oklahoma City Thunder, Golden State Warriors, Chicago Bulls, and Denver Nuggets. So "New York Knicks", "Boston Celtics", "Oklahoma City Thunder", "Golden State Warriors", "Chicago Bulls", and "Denver Nuggets" is the answer of question "Which team does Nate Robinson play for?". Now, for step 3, I need to find the intersection of step 1 and step 2. "Denver Nuggets" is the step 1's answer and "New York Knicks", "Boston Celtics", "Oklahoma City Thunder", "Golden State Warriors", "Chicago Bulls", and "Denver Nuggets" are the step 2's function. So the intersection of step 1 and step 2 is "Denver Nuggets". `</think>`

`<answer>`Denver Nuggets`</answer>`

Figure 8: Case Study.

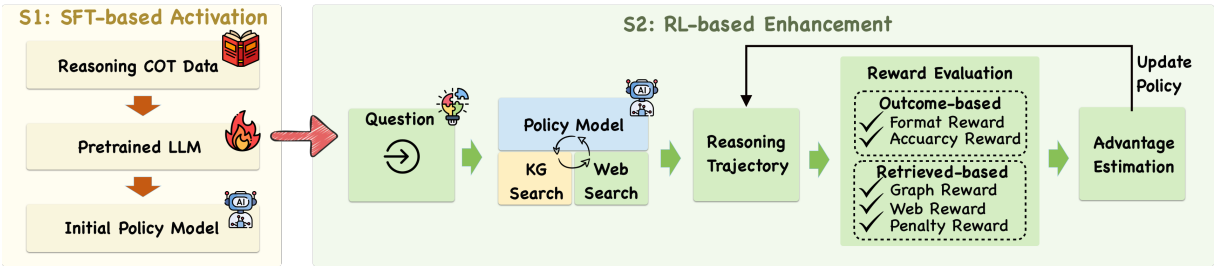


Figure 5: Graph-RFT Training Framework: SFT-based Reasoning Activation (S1) and RL-based Reasoning Enhancement (S2).

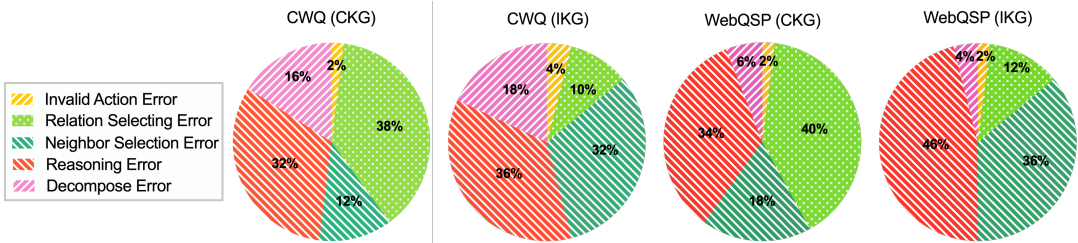


Figure 6: Error Type